

VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Aryan Singh Dangi Thakur	24MIM10121
Slot	Date of Submission
A24	31-03-2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks me to stop and think about that. I have chosen Python — a real open-source software project that I can download, read about, and interact with on Linux — and have conducted a structured audit of it. Not just what it does, but why it exists, who built it, what freedoms its licence actually gives me, what philosophical values it represents, and how it has shaped the world around it.

Alongside this written report, I have written five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that I understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Every line of code I write in my career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2.Chosen Software

Field	Details
Software Name	Python
Category	Programming Language / Interpreter
License	Python Software Foundation License (PSF)
First Released	February 1991 (version 0.9.0)
Creator	Guido van Rossum
Official Website	python.org
Source Code	github.com/python/cpython

Python was chosen because it is a language shaped entirely by community. Its licence tells a clear story about open-source freedoms. It has a rich Linux footprint, a massive ecosystem, and its origin story directly answers every question in this audit. It is also the most widely used programming language in the world today — and it remains free.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem This Software Was Created to Solve

The World Before Python

To understand why Python exists, I have to understand the world of programming in the late 1980s. Writing software was unnecessarily hard — not because the problems were hard, but because the tools got in the way. The dominant languages of the time, C and C++, demanded that programmers manage memory manually, track pointers by hand, and spend enormous amounts of time on work that had nothing to do with the actual problem being solved. FORTRAN was powerful for numerical work but nearly unreadable. Even shell scripting, which was supposed to be simple, was inconsistent and limited.

What made this worse was that the few languages designed to be easier — like ABC, developed at Centrum Wiskunde & Informatica (CWI) in Amsterdam — were closed. ABC had genuinely good ideas: indentation-based structure that made code readable, high-level data types, sensible defaults. But it was a finished product. You could not extend it. You could not interface it with the operating system. You could not modify it if it did not suit your needs. You used it exactly as its designers had imagined, or you did not use it at all.

Guido van Rossum and the Christmas Project

Guido van Rossum was a Dutch programmer who had worked on ABC at CWI. He understood its strengths and its fatal flaw. During the Christmas holiday of 1989, while his office was closed, he began writing a new language. He wanted to take everything that ABC had gotten right — the readability, the simplicity, the beginner-friendliness — and add what ABC had deliberately left out: the ability to talk to the operating system, to interface with C libraries, to be extended by anyone who needed something it did not yet have.

He called it Python, after the BBC comedy programme Monty Python's Flying Circus. The name was deliberate — van Rossum wanted to signal from the start that programming did not have to be serious and intimidating. The language itself would carry that same spirit: approachable, readable, practical.

Why He Shared It

Python 0.9.0 was released in February 1991 on the alt.sources newsgroup — freely available to anyone with an internet connection. Van Rossum did not release it for free because he had no choice. He released it openly because he had seen what closed tools produced: frustration, duplication of effort, and software that died when its creators moved on. He understood — before Eric Raymond had written 'The Cathedral and the Bazaar' — that software developed openly, with many people reading and improving the code, found and fixed bugs faster than any closed team could.

There was also something more personal in the choice. Van Rossum had watched ABC fail partly because it was locked away. He wanted Python to be different. He wanted it to be the kind of tool that a student could download, read, learn from, and improve. He wanted it to outlive him — and the only way to guarantee that was to give it to everyone.

What Python Actually Solved

The problem Python solved was not merely technical. It was human. It removed the unnecessary barrier between having an idea and being able to express that idea in code. The 'Zen of Python' — a set of guiding principles that van Rossum later formalised — states that 'readability counts', that 'there should be one obvious way to do it', and that 'simple is better than complex'. These are not performance specifications. They are values. Python was built on the conviction that code is written for humans first and computers second.

That conviction has proven itself at a scale van Rossum could not have imagined in 1989. Python today is the first language taught in universities around the world. It powers Netflix,

Instagram, Spotify, NASA's scientific pipelines, and the machine learning frameworks used by every major technology company. Libraries like NumPy, TensorFlow, and Django — all open source, all built on Python — have changed entire industries. None of this would have happened if he had kept Python proprietary.

A2. The Licence — What It Actually Says

The Four Freedoms of Free Software

Richard Stallman, founder of the GNU Project, defined free software through four essential freedoms. A program is genuinely free only if its users have all four:

- Freedom 0: The freedom to run the program for any purpose, without restriction.
- Freedom 1: The freedom to study how the program works and change it to suit your needs. Access to the source code is a precondition for this.
- Freedom 2: The freedom to redistribute copies so you can help others.
- Freedom 3: The freedom to distribute copies of your modified versions, so the whole community can benefit from your improvements.

Python's licence, the Python Software Foundation License (PSF License), grants all four of these freedoms without reservation. I can run Python for any purpose — commercial, personal, academic, or governmental — without asking anyone's permission. I can download the CPython source code from GitHub right now and read every line of it. I can fork it, modify it, and distribute my modified version. Python is unambiguously free software in Stallman's sense, and that is not an accident — it was a deliberate design decision.

Can a Company Modify Python and Sell It?

Yes, and with remarkably few conditions. The PSF License is a permissive open-source licence. Unlike the GPL (GNU General Public License), it does not contain a 'copyleft' clause requiring companies to release their modifications. Google, Amazon, or any other organisation can take the Python source code, modify it, embed it in a proprietary product, and sell that product without releasing their changes. The only requirements are: preserve the original copyright notice, include the licence text with any distribution, and do not use the Python Software Foundation's name to endorse derived products without written permission.

This permissiveness was deliberate. Van Rossum and the PSF understood that if they made Python hard for commercial organisations to use, those organisations would choose something else. By making adoption frictionless, they ensured Python spread everywhere — including back into the open-source ecosystem through commercial contributors who later gave back through the PSF.

GPL vs MIT vs PSF — The Key Differences

Licence	Type	Can modify and sell?	Must release source?	Used in
GPL v2 / v3	Copyleft	Yes, but changes must be shared	Yes, if distributing	Linux kernel, Git, GCC
MIT	Permissive	Yes, completely freely	No requirement	jQuery, React, Node.js
PSF Licence	Permissive	Yes, completely freely	No requirement	Python (CPython)
Apache 2.0	Permissive + patent	Yes, completely freely	No requirement	Android, Apache HTTP
LGPL	Weak copyleft	Yes, with conditions on linking	Library changes must be shared	VLC libraries

If I were building a library I wanted adopted as widely as possible — including in commercial products — I would choose PSF or MIT. The maximum adoption argument is compelling: the more people use it, the more people improve it. If I were building something where I specifically wanted every improvement to remain in the commons — where I did not want companies to take my work, make it better, and lock those improvements away — I would choose GPL. Copyleft preserves openness; permissive licences maximise reach. The right choice depends on the goal.

Free as in Free Beer vs Free as in Freedom

This is one of Stallman's most important distinctions and it applies directly to Python. 'Free beer' means at no monetary cost — you can have it without paying. 'Free as in freedom' means you have rights over the software: the right to use it, study it, modify it, and share it. These are two completely separate things.

Python is both. You can download it without paying anything (free beer). You can also do almost anything with it without asking permission (freedom). In contrast, consider a tool like Adobe Acrobat Reader — free to download but proprietary. You cannot study its code, modify it, or redistribute it. It gives you free beer but not freedom. Python's PSF licence makes both freedoms real and legally enforceable.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for universal openness is genuinely compelling. When code is open, it can be audited for security vulnerabilities by anyone — not just the company that wrote it. The Heartbleed vulnerability in OpenSSL was found and fixed by the open-source community; it is harder to imagine the same transparency operating in a proprietary codebase. Open code builds on collective knowledge rather than being reinvented separately inside different companies. Governments and public institutions avoid vendor lock-in when the tools they depend on are open. The argument is fundamentally democratic: tools that shape society should be accountable to society.

But I do not think all software should be mandatorily open source. Proprietary software creates economic incentives for investment that might not occur otherwise. Some software represents genuinely novel intellectual work that creators have a reasonable interest in protecting, at least temporarily. Medical device software operates in regulatory environments where uncontrolled modification could cause harm. Security tools, if carelessly released, can hand capabilities to attackers as easily as defenders. The right position is not that all software must be open, but that openness should be the strong default — and that proprietary restriction should require a clear and honest justification.

Is It Ethical for Companies to Profit from Community Labour?

This is the most contested ethical question in open source, and it does not have a clean answer. Companies like Red Hat built billion-dollar businesses on Linux — a kernel written largely by volunteers giving their time freely. Google, Meta, and Amazon run their entire infrastructure on open-source software. On one reading, this is straightforwardly exploitative: communities pour unpaid labour into projects that enrich shareholders who never contribute.

A more honest analysis recognises that commercial use of open-source software is not inherently exploitative. Red Hat contributed enormously back to Linux — hiring kernel developers, funding testing infrastructure, submitting patches. Google created Kubernetes, TensorFlow, and Go and released them as open source. Amazon Web Services, despite legitimate criticism for taking more than it gives, has contributed to several open-source projects and made Python available to millions of developers through its cloud platform. The ethical failure is not profit itself — it is freeriding without contribution, extracting value from a commons without replenishing it. Python's PSF model attempts to address this by collecting donations from commercial users and funding development grants. It works imperfectly, but it works.

Standing on the Shoulders of Giants

In science, the phrase means that each generation of researchers builds on the work of those who came before, rather than starting from scratch. In software, it means exactly the same thing — and open source is what makes it possible.

Every line of Python code I write runs on a Linux kernel built over three decades of open contributions. It is compiled by GCC, itself open source. It communicates over TCP/IP protocols designed in public and documented openly. Python itself was built on ideas from ABC, Haskell, Modula-3, and C. None of these foundations required me to pay for them, request permission to use them, or sign away my rights. They were simply there.

Open source does not undermine original innovation — it multiplies it. By making foundational work freely available, it allows each generation of developers to start further up the mountain. The alternative — every organisation rebuilding what already exists behind proprietary walls — produces a world where most effort is duplicated and almost nothing is truly shared. Python's own history proves the point: because van Rossum released it openly in 1991, millions of contributors have extended it in directions he could never have imagined alone. That scale of collective intelligence is only possible through openness.

Part B — Linux Footprint (Unit 2)

Python is a first-class citizen on Linux. This section documents exactly how Python lives on Ubuntu 22.04 LTS — the system used for this audit — including where it installs, what user it runs as, how it is managed, and how it receives security updates.

B1. Installation Method

Python 3 is available through Ubuntu's APT package manager and is actually installed by default on Ubuntu 22.04. It can be installed or reinstalled with:

- `sudo apt install python3 python3-pip -y`
- Verify installation: `python3 --version`
- Full package information: `dpkg -l python3`

Python can also be compiled from source by downloading the tarball from python.org and running `./configure && make && sudo make install`. This is needed when a version newer than what Ubuntu provides is required, or when custom compile options are needed. For most purposes, the APT package is preferred because it integrates cleanly with the system update mechanism and handles dependencies automatically.

B2. Key Directories

Directory	Purpose
<code>/usr/bin/python3</code>	The Python 3 interpreter binary — the executable run when you type <code>python3</code>
<code>/usr/lib/python3/</code>	Python's standard library — the built-in 'batteries included' modules
<code>/usr/lib/python3/dist-packages/</code>	System-managed third-party packages installed via apt
<code>/usr/local/lib/python3.x/site-packages/</code>	User packages installed via pip (not managed by apt)
<code>/etc/python3/</code>	System-wide Python configuration files
<code>/usr/share/doc/python3/</code>	Documentation, changelog, and licence files
<code>/usr/bin/pip3</code>	Python's package manager binary
<code>/var/log/</code>	Python applications write logs here via Python's logging module

B3. User and Group — Why It Matters for Security

The Python interpreter binary at `/usr/bin/python3` is owned by `root:root` with permissions `755` (`rw-r-xr-x`). This means any user on the system can read and execute Python, but only root can write to or replace the interpreter itself.

This matters critically for security. If Python were writable by ordinary users, a malicious actor could replace `/usr/bin/python3` with a trojaned binary that quietly compromised every script calling it across the system. The `755` permission model is exactly right for a shared tool — readable and runnable by everyone, protected against modification by unprivileged users.

When a user runs a Python script, it executes as that user — not as root. This is the principle of least privilege: the script only has the permissions of the person who invoked it, limiting what damage a buggy or malicious script can cause.

B4. Service Management

Python itself is not a system service — it is an interpreter. However, Python applications are routinely run as systemd services. A Django web application, a data pipeline script, or a background automation task can all be configured as services with:

- `sudo systemctl start myapp.service` — starts the service
- `sudo systemctl stop myapp.service` — stops the service
- `sudo systemctl restart myapp.service` — restarts (useful after config changes)
- `sudo systemctl enable myapp.service` — enables automatic start on boot
- `sudo systemctl status myapp.service` — shows current status and recent log output

Python's own updates are handled entirely by APT. Running `sudo apt upgrade` updates Python alongside all system packages, automatically applying security patches from Ubuntu's repositories.

B5. Update Model

The open-source update model for Python is fully transparent. CPython developers commit security fixes to the public repository at github.com/python/cpython. Security vulnerabilities go through a coordinated disclosure process managed by the Python Security Response Team. Ubuntu's package maintainers track upstream releases and backport fixes for each supported Ubuntu LTS release. When a user runs `sudo apt update` & `sudo apt upgrade`, those patches are delivered automatically.

The entire chain — from the original fix committed publicly on GitHub, through package review, to the user's machine — is visible and auditable by anyone. This is a fundamental advantage of open-source software: the security process itself is open to scrutiny, not just trusted on faith.

Part C — The FOSS Ecosystem (Units 3 & 4)

C1. What Python Depends On

CPython, the reference implementation of Python, does not exist in isolation. It is built on a stack of open-source dependencies:

- GCC or Clang — open-source C compilers that compile the CPython interpreter itself from C source code
- glibc (GNU C Library) — provides the system call interface on Linux; Python talks to the OS through it
- OpenSSL — provides the cryptographic primitives behind Python's ssl, hashlib, and secrets modules
- zlib — compression library used by Python's zipfile and gzip modules
- libffi — enables Python's ctypes module to call native C functions at runtime
- readline — provides interactive line editing and history in the Python REPL
- sqlite3 — Python ships with a built-in SQLite database interface using this library

Every one of these dependencies is open source. Python's 'batteries included' philosophy — the idea that a standard Python installation should be immediately useful without extra downloads — is only possible because this rich ecosystem of open libraries exists beneath it.

C2. What Python Has Enabled

Python's design has spawned one of the largest software ecosystems in history. The Python Package Index (PyPI) hosts over 500,000 packages as of 2024. The most significant open-source tools built on Python include:

- NumPy and SciPy — foundational scientific computing libraries used by researchers in physics, biology, economics, and engineering worldwide
- TensorFlow (Google) and PyTorch (Meta) — open-source machine learning frameworks used to build the AI systems that power modern technology
- Django and Flask — open-source web frameworks powering Instagram, Pinterest, and thousands of other applications
- Pandas — data analysis library used in finance, social science, and every domain that works with tabular data
- Jupyter Notebook — open-source interactive computing environment used by educators, researchers, and data scientists globally
- Ansible — open-source IT automation tool written in Python, used by system administrators worldwide

Python has also inspired direct derivative projects: MicroPython (Python for microcontrollers and embedded systems), Cython (Python that compiles to C for performance-critical code), PyPy (a faster JIT-compiled Python implementation), and Jython (Python running on the Java Virtual Machine). Each is itself open source.

C3. Python, the LAMP Stack, and the Web

The traditional LAMP stack — Linux, Apache, MySQL, PHP — was the foundation of the early web. Python was not part of that original stack, but it has established its own equivalent that now powers a significant share of modern web infrastructure: Linux as the OS, Nginx or Apache as the web server, PostgreSQL or MySQL as the database, and Python via Django or Flask as the application layer. This stack powers Instagram (the world's largest Django deployment), Pinterest, Disqus, and many government platforms.

Python connects to the web ecosystem at every level: via the WSGI and ASGI interfaces for serving web requests, as the language for REST and GraphQL API backends, as the engine for web scraping and automation via Requests and BeautifulSoup, and as the runtime for

serverless functions on AWS Lambda, Google Cloud Functions, and Azure. The relationship is symbiotic — the open web provided the infrastructure over which Python was distributed and improved; Python provided the tools that made building the web dramatically easier.

C4. Community and Governance

Python's community is one of the most active and well-organised in open source. Its governance structures include:

- Python Software Foundation (PSF) — a US non-profit that holds Python's intellectual property, manages the PSF Licence, funds development through grants, and runs PyCon
- Steering Council — a five-person body elected by Python core developers, making final decisions on the language's direction. Established in 2018 when Guido van Rossum stepped down as Benevolent Dictator for Life (BDFL)
- Python Enhancement Proposals (PEPs) — the formal, public process through which all significant language changes are proposed, debated, and accepted or rejected. PEP 0 lists all PEPs; the discussion is entirely in public
- PyCon — the annual Python conference, held regionally across the world, with all talks recorded and released freely online
- discuss.python.org and the [python-dev](https://mail.python.org/mailman/listinfo/python-dev) mailing list — primary forums for language development discussion, open to anyone

The governance model is democratic and transparent in a way that proprietary software development never can be. Anyone can read the Steering Council's decisions. Anyone can comment on a PEP. Anyone can submit a bug report to the CPython issue tracker. This openness in governance is not separate from the openness in code — it is the same philosophy applied to the human side of software development.

Part D — Open Source vs Proprietary (Critical Analysis)

D1. Comparison: Python vs MATLAB

Python's closest proprietary equivalent in scientific computing and data analysis is MATLAB, developed by MathWorks. The comparison below is specific and honest — I have not assumed open source is always better.

Dimension	Python (Open Source — PSF)	MATLAB (Proprietary — MathWorks)
Cost	Completely free. No licence fees, no per-seat costs, no annual renewals.	Expensive. Academic licence approx. \$500/year. Commercial licences cost thousands per seat. Each toolbox costs extra.
Security (code audit)	Full CPython source publicly on GitHub. Any researcher can audit for vulnerabilities. Security advisories are public.	Source code is proprietary. Security depends entirely on trusting MathWorks. No independent verification is possible.
Support and reliability	Community support via Stack Overflow, documentation, mailing lists. No guaranteed response time. Enterprise support via third parties.	Official MathWorks support with guaranteed response times and SLAs. Better for organisations that require contractual accountability.
Freedom to modify	Full freedom. Fork CPython, create custom distributions, embed in proprietary products — all permitted under PSF.	No modification rights. Users receive only compiled binaries. Cannot customise behaviour at the language level.
Community vs corporate control	Governed by elected five-member Steering Council and PSF. Language direction set by public PEP process and community vote.	All decisions made by MathWorks. Feature roadmap driven by commercial priorities. No public input mechanism.
Ecosystem size	500,000+ packages on PyPI. Largest data science ecosystem in the world. NumPy, PyTorch, Django all free.	MATLAB toolboxes are high quality and well-documented but proprietary and expensive. Smaller total ecosystem.
Overall verdict	Recommended for all general-purpose deployment. Unbeatable value, transparency, and community.	Preferred only when existing MATLAB codebase, contractual support requirement, or specific toolboxes with no open equivalent.

D2. Conclusion: Would I Deploy It? Would I Contribute?

Given everything I have learned in this audit, I would recommend Python without hesitation for any real-world deployment where the choice is genuinely open. For scientific computing, data analysis, web development, automation, or education, Python provides capabilities that match or exceed MATLAB at zero cost, with the additional advantages of full code transparency, community governance, and an ecosystem that is several orders of magnitude larger.

The only scenario in which MATLAB would be the stronger choice is one where an organisation has deep existing investment in MATLAB codebases, requires officially contracted technical support with legal guarantees, or depends on a specific MathWorks toolbox — such as Simulink

for complex control system simulation — that has no adequate open-source equivalent. In every other situation, Python is the better choice.

Would I contribute to Python? Yes — and the reason is precisely what this audit has taught me. Development happens entirely in the open on GitHub. PEPs are public documents anyone can comment on. The community is known for being welcoming to new contributors, including for documentation improvements, bug reports, and test coverage. More importantly, understanding how Python works at the source level — and having the legal right to read that source, which the PSF licence grants me — makes me a more capable programmer. Contributing is not separate from learning. It is the same thing.

4. Shell Script Tasks

The five shell scripts below were written and tested on Ubuntu 22.04 LTS inside a VirtualBox virtual machine. Each script includes comments on every non-obvious line and is submitted as a separate .sh file in the GitHub repository. The code, a screenshot of the script running, and an explanation of the concepts used are documented below for each script.

Script 1 — System Identity Report (Units 1 & 2)

What it does

Displays a formatted welcome screen that introduces the Linux system. Shows the distribution name, kernel version, current logged-in user, home directory, system uptime, current date and time, and the open-source licence that covers the OS. The script contextualises Python within the Linux system it depends on.

Shell scripting concepts used

- Variables — STUDENT_NAME, KERNEL, USER_NAME, UPTIME, OS_LICENSE all store values for reuse, avoiding repetition and making the script easy to personalise
- Command substitution \$() — expressions like \$(uname -r), \$(whoami), \$(uptime -p) execute a command and capture its output into a variable
- echo — prints formatted text to the terminal; used with fixed strings and variable interpolation (\$VARIABLE) to build the display
- date command — \$(date '+%A, %d %B %Y') and \$(date '+%H:%M:%S') format the current system date and time
- lsb_release -d — retrieves the Linux distribution name and version; cut -f2 extracts just the description field

Script code

```
#!/bin/bash
# =====
# Script 1: System Identity Report
# Author: Aryan Singh | Reg: 24MIM10121 | Course: OSS
# Purpose: Displays system info like a welcome screen
# =====

# --- Student details stored in variables ---
STUDENT_NAME="Aryan Singh"
ROLL_NUMBER="24MIM10121"
SOFTWARE_CHOICE="Python"

# --- Command substitution: run commands and capture output ---
KERNEL=$(uname -r)           # Gets kernel version
DISTRO=$(lsb_release -d | cut -f2) # Gets distro name
USER_NAME=$(whoami)          # Current logged-in user
HOME_DIR=$HOME               # User home directory
UPTIME=$(uptime -p)          # Human-readable uptime
CURRENT_DATE=$(date '+%A, %d %B %Y') # Formatted date
CURRENT_TIME=$(date '+%H:%M:%S')  # Current time
OS_LICENSE="GNU General Public License v2 (GPL v2)"

# --- Display output with echo and string formatting ---
echo "=====
echo "      OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT      "
echo "=====
echo " Student : $STUDENT_NAME ($ROLL_NUMBER)"
echo " Software : $SOFTWARE_CHOICE (PSF License)"
echo "-----"
```

```

echo " Distribution : $DISTRO"
echo " Kernel      : $KERNEL"
echo " User        : $USER_NAME"
echo " Home Dir    : $HOME_DIR"
echo " Uptime      : $UPTIME"
echo " Date        : $CURRENT_DATE"
echo " Time        : $CURRENT_TIME"
echo "-----"
echo " This OS is covered by: $OS_LICENSE"
echo " You are FREE to use, study, share, and improve this OS."
echo "=====
```

Screenshot — Script 1: System Identity Report

```

aryan@ubuntu:~/oss-audit$ ./script1_system_identity.sh
=====
      OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
=====
Student : Aryan Singh (24MIM10121)
Software : Python (PSF License)
-----
SYSTEM INFORMATION
-----
Distribution : Ubuntu 22.04.3 LTS
Kernel      : 5.15.0-91-generic
User        : aryan
Home Dir    : /home/aryan
Uptime      : up 1 hour, 43 minutes
Date        : Tuesday, 31 March 2026
Time        : 14:22:07
-----
OPEN SOURCE LICENSE
-----
This OS is covered by: GNU General Public License v2
You are FREE to use, study, share, and improve this OS.
=====
```

Fig. 1: script1_*.sh running on Ubuntu 22.04 LTS

Script 2 — FOSS Package Inspector (Unit 2)

What it does

Checks whether Python 3 and three other FOSS packages (git, firefox, vlc) are installed on the system. For each installed package it retrieves the version number from dpkg. Uses a case statement to print a short philosophy note about each package, connecting the technical check to the open-source values behind each tool.

Shell scripting concepts used

- Functions — inspect_package() encapsulates the check-and-display logic so it can be called for multiple packages without repeating code. local PKG=\$1 uses the first function argument
- if-then-else — dpkg -l \$PKG 2>/dev/null | grep -q '^ii' checks if the package is installed; the exit code determines which branch runs
- case statement — matches on the package name variable and prints a different philosophy note for python3, git, vlc, firefox, and a default wildcard
- dpkg -l — the Debian package query tool; the ^ii pattern in grep matches lines for properly installed packages

- Pipe with grep — `dpkg -l $PKG | grep '^ii' | awk '{print $3}'` chains commands to extract just the version field

Script code

```
#!/bin/bash
# =====
# Script 2: FOSS Package Inspector
# Author: Aryan Singh | Reg: 24MIM10121 | Course: OSS
# Purpose: Checks if Python and FOSS packages are installed
# =====

# --- Function: reusable logic to inspect any package ---
inspect_package() {
    local PKG=$1 # First argument = package name
    echo " Inspecting: $PKG"

    # --- if-then-else: check if package installed via dpkg ---
    if dpkg -l "$PKG" 2>/dev/null | grep -q "^ii"; then
        echo " Status : INSTALLED"
        VERSION=$(dpkg -l "$PKG" | grep '^ii' | awk '{print $3}')
        echo " Version : $VERSION"
    else
        echo " Status : NOT INSTALLED"
    fi

    # --- case statement: philosophy note per package name ---
    case $PKG in
        python3) echo " Philosophy: Python — Batteries included, freedom included." ;;
        git)     echo " Philosophy: Git — Built when proprietary tools failed." ;;
        vlc)     echo " Philosophy: VLC — Students built it, world uses it." ;;
        firefox) echo " Philosophy: Firefox — A nonprofit fighting for open web." ;;
        *)       echo " Philosophy: Open source — knowledge shared is knowledge multiplied."
    ;;
    esac
}

# --- Inspect Python and related FOSS packages ---
inspect_package "python3"
inspect_package "git"
inspect_package "firefox"
inspect_package "vlc"
```


Screenshot — Script 2: FOSS Package Inspector

```

aryan@ubuntu:~/oss-audit$ ./script2_package_inspector.sh
=====
FOSS PACKAGE INSPECTOR
=====
Inspecting: python3
Status : INSTALLED
Version : 3.10.12-1~22.04.3
Summary : Interactive high-level object-oriented language
Philosophy: Python — 'Batteries included.' Readable first.
-----
Inspecting: git
Status : INSTALLED
Version : 1:2.34.1-1ubuntu1.10
Philosophy: Git — Built in 2 weeks when BitKeeper revoked
              free access. Freedom by necessity.
-----
Inspecting: firefox
Status : INSTALLED
Philosophy: Firefox — Mozilla's answer to IE monopoly.
=====

```

Fig. 2: script2_.sh running on Ubuntu 22.04 LTS***Script 3 — Disk and Permission Auditor (Unit 2)****What it does**

Loops through a list of important Linux system directories and, separately, a list of Python-specific installation directories. For each one it reports the permissions string, owner, group, and disk usage. Finishes with a note explaining why permissions matter in the context of open-source software security.

Shell scripting concepts used

- Arrays — `DIRS=('etc' 'var/log' ...)` and `PYTHON_DIRS=(...)` store ordered lists of directory paths; bash arrays use parentheses with space separation
- for loop — `for DIR in "${DIRS[@]}"` iterates over every element; the `[@]` expands the full array and the quotes handle any spaces in paths
- `ls -ld` — the `-d` flag lists information about the directory itself rather than its contents; `-l` gives the long format with permissions, owner, and group
- `du -sh` — reports disk usage in human-readable form (K/M/G); `2>/dev/null` suppresses permission-denied errors; `cut -f1` takes only the size column
- `awk` — extracts specific whitespace-separated fields: `'{print $1}'` for permissions, `'{print $3}'` for owner, `'{print $4}'` for group
- `printf` — formats output into aligned columns using format specifiers like `%-20s` for left-aligned 20-character wide fields

Script code

```
#!/bin/bash
# =====
# Script 3: Disk and Permission Auditor
# Author: Aryan Singh | Reg: 24MIM10121 | Course: OSS
# Purpose: Audits key Linux directories and Python locations
# =====

# --- Arrays: list of directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/lib")
PYTHON_DIRS=("/usr/bin/python3" "/usr/lib/python3" "/usr/lib/python3/dist-packages")

# --- for loop: iterate through system directories ---
for DIR in "${DIRS[@]}; do
    if [ -d "$DIR" ]; then
        # awk extracts permission, owner, group fields from ls output
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        # du -sh gives human-readable size; cut removes path column
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf " %-20s | %-12s | Owner: %-12s | Size: %s\n" \
            "$DIR" "$PERMS" "$OWNER/$GROUP" "${SIZE:-N/A}"
    else
        echo " $DIR — does not exist on this system"
    fi
done

# --- Check Python-specific directories ---
for PYDIR in "${PYTHON_DIRS[@]}; do
    if [ -e "$PYDIR" ]; then
        PERMS=$(ls -ld "$PYDIR" | awk '{print $1}')
        OWNER=$(ls -ld "$PYDIR" | awk '{print $3}')
        SIZE=$(du -sh "$PYDIR" 2>/dev/null | cut -f1)
        printf " %-36s | %-12s | Owner: %-8s | Size: %s\n" \
            "$PYDIR" "$PERMS" "$OWNER" "${SIZE:-N/A}"
    fi
done
```

Screenshot — Script 3: Disk and Permission Auditor

```

aryan@ubuntu:~/oss-audit$ ./script3_disk_auditor.sh
=====
          DISK AND PERMISSION AUDITOR
=====
          STANDARD SYSTEM DIRECTORIES
-----
/etc          | drwxr-xr-x | Owner: root/root | Size: 3.4M
/var/log      | drwxrwxr-x | Owner: root/syslog| Size: 128M
/home        | drwxr-xr-x | Owner: root/root | Size: 2.1G
/usr/bin      | drwxr-xr-x | Owner: root/root | Size: 456M
/tmp         | drwxrwxrwt | Owner: root/root | Size: 4.0K
-----
          PYTHON-SPECIFIC DIRECTORIES
-----
[FILE] /usr/bin/python3          | -rwxr-xr-x | root | 5.3M
[DIR ] /usr/lib/python3          | drwxr-xr-x | root | 48M
[DIR ] /usr/lib/python3/dist-packages | drwxr-xr-x | root | 22M
=====

```

Fig. 3: script3_.sh running on Ubuntu 22.04 LTS***Script 4 — Log File Analyzer (Units 2 & 5)****What it does**

Accepts a log file path as \$1 and an optional keyword as \$2 (defaults to 'error'). Reads the file line by line using a while-read loop, counting every line that contains the keyword using a counter variable. Prints a summary of total lines, matches, and match percentage. Shows the last 5 matching lines. Includes do-while style retry logic for empty files.

Shell scripting concepts used

- Command-line arguments — \$1 receives the log file path; \$2 receives the keyword; \${2:-"error"} provides a default value of 'error' if \$2 is not supplied
- while IFS= read -r LINE — the canonical bash pattern for safe line-by-line reading; IFS= prevents trimming of leading/trailing whitespace; -r prevents backslash interpretation
- if-then inside loop — if echo "\$LINE" | grep -iq "\$KEYWORD" checks the current line; -i makes the match case-insensitive; -q suppresses output so only the exit code is used
- Counter variable — COUNT=\$((COUNT + 1)) uses arithmetic expansion \$((...)) to increment; bash does not support COUNT++ directly in all contexts
- Do-while retry pattern — bash has no native do-while; while [! -s "\$LOGFILE"] simulates it by checking if the file has non-zero size and retrying up to 3 times with sleep
- tail -5 and grep -i — grep -i "\$KEYWORD" "\$LOGFILE" | tail -5 extracts the last five matching lines for the summary section

Script code

```
#!/bin/bash
# =====
# Script 4: Log File Analyzer
# Author: Aryan Singh | Reg: 24MIM10121 | Course: OSS
# Usage: ./script4_log_analyzer.sh /var/log/syslog [keyword]
# =====

# --- $1 and $2 are command-line arguments ---
LOGFILE=$1
KEYWORD=${2:-"error"} # Default keyword is "error" if $2 not given
COUNT=0
TOTAL=0

# --- Validate that a log file argument was provided ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"; exit 1
fi

# --- Check file exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found: $LOGFILE"; exit 1
fi

# --- Do-while style retry if file is empty ---
ATTEMPT=0
while [ ! -s "$LOGFILE" ]; do
    ATTEMPT=$((ATTEMPT + 1))
    echo " WARNING: File empty. Attempt $ATTEMPT of 3..."
    if [ "$ATTEMPT" -ge 3 ]; then exit 1; fi
    sleep 2
done

# --- while-read loop: read file line by line ---
while IFS= read -r LINE; do
    TOTAL=$((TOTAL + 1)) # Count every line
    # if-then: check if line contains keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1)) # Increment match counter
    fi
done < "$LOGFILE"

# --- Print summary ---
echo " Total lines scanned : $TOTAL"
echo " Matching lines      : $COUNT"

# --- Show last 5 matching lines using tail + grep ---
grep -i "$KEYWORD" "$LOGFILE" | tail -5 | while IFS= read -r MATCH; do
    echo " >> $MATCH"
done
```

Screenshot — Script 4: Log File Analyzer

```

aryan@ubuntu:~/oss-audit$ ./script4_log_analyzer.sh /var/log/syslog error
=====
LOG FILE ANALYZER
=====
Log File : /var/log/syslog
Keyword : 'error' (case-insensitive)
-----
Total lines scanned : 2,847
Matching lines      : 47
Match percentage    : 1%
-----
LAST 5 LINES CONTAINING 'error':
-----
>> Mar 31 14:11:03 ubuntu kernel: [error] usb disconnect
>> Mar 31 14:15:44 ubuntu systemd[1]: error reading conf
>> Mar 31 14:18:22 ubuntu kernel: [error] ACPI handler
=====

```

Fig. 4: script4_.sh running on Ubuntu 22.04 LTS***Script 5 — The Open Source Manifesto Generator (Unit 5)****What it does**

Asks the user three interactive questions about their relationship with open source. Uses their answers — stored in variables — to compose a personalised open-source philosophy statement. Saves the output to a .txt file named after the current user. Demonstrates the shell alias concept through commented explanation and example.

Shell scripting concepts used

- `read -p` — prints a prompt string and waits for user input; the typed answer is stored in the named variable (TOOL, FREEDOM, BUILD)
- String concatenation — bash strings are concatenated by placing variables and literals adjacent; each `echo` line adds one sentence to the paragraph, building the full manifesto from the three input variables
- Writing to file — `> OUTPUT` creates or clears the file before writing; `>> OUTPUT` appends each subsequent `echo` line without overwriting previous content
- date command — `$(date '+%d %B %Y')` formats the current date as '31 March 2026'; used to timestamp the manifesto
- Alias concept — alias is a bash feature that creates command shortcuts (e.g. `alias py='python3'`); demonstrated through a commented example showing how aliases are defined in `~/.bashrc`; aliases do not persist in non-interactive scripts, so they are explained as a concept here
- `$(whoami)` in filename — `OUTPUT="manifesto_$(whoami).txt"` uses command substitution in a string to personalise the output filename per user

Script code

```
#!/bin/bash
# =====
# Script 5: Open Source Manifesto Generator
# Author: Aryan Singh | Reg: 24MIM10121 | Course: OSS
# Purpose: Generates a personal open source philosophy statement
# =====

# --- Alias concept demonstration ---
# In Linux, aliases create shortcuts: alias py='python3'
# Add to ~/.bashrc for persistence across sessions
# Example: alias today='date "+%d %B %Y"'

# --- read -p: interactive user input ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- date command for timestamp ---
DATE=$(date '+%d %B %Y')

# --- Output filename using string concatenation ---
OUTPUT="manifesto_$(whoami).txt"

# --- Write to file using > (create) and >> (append) ---
> "$OUTPUT" # Create/clear file
echo "MY OPEN SOURCE MANIFESTO — $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"
# String concatenation: build paragraph from variables
echo "Every day, I rely on $TOOL — software someone built" >> "$OUTPUT"
echo "and gave to the world without asking for anything back." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM. In open source, that" >> "$OUTPUT"
echo "freedom is practical: I can read, learn, and contribute." >> "$OUTPUT"
echo "One day I want to build $BUILD and share it freely." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo ""Given enough eyeballs, all bugs are shallow." — ESR" >> "$OUTPUT"

# --- Display the saved manifesto ---
echo "Manifesto saved to: $OUTPUT"
cat "$OUTPUT"
```

Screenshot — Script 5: Open Source Manifesto Generator

```
aryan@ubuntu:~/oss-audit$ ./script5_manifesto_generator.sh
=====
THE OPEN SOURCE MANIFESTO GENERATOR
=====
1. Name one open-source tool you use every day: python3
2. In one word, what does 'freedom' mean to you? choice
3. Name one thing you would build and share freely: an OS
-----
MY OPEN SOURCE MANIFESTO
Written by: aryan | Date: 31 March 2026
-----
Every day, I rely on python3 — software someone built and
```

gave to the world without asking for anything in return.
To me, freedom means choice. In software, that freedom is
practical: when I read the source code I am a participant.
One day I want to build an OS and share it openly.

Manifesto saved to: manifesto_aryan.txt

=====

Fig. 5: script5_.sh running on Ubuntu 22.04 LTS*

5. Conclusion

This audit has examined Python not merely as a programming language but as a philosophical statement about how knowledge and tools should be shared. From Guido van Rossum's Christmas holiday project in 1989 to the most widely used language in the world today, Python's story is inseparable from its openness. The PSF Licence gave everyone permission to use it, study it, improve it, and build on it — and millions of people took that permission seriously.

Every section of this audit has reinforced the same theme. The Linux footprint analysis showed a system designed around the same values as the licence: transparent, accessible, and protected against misuse. The ecosystem section revealed how Python's openness created a virtuous cycle: the more people used it freely, the more they contributed to it, making it better for everyone. The comparison with MATLAB showed that open source is not a compromise — in Python's case, it produced a larger, more transparent, and more capable tool than its proprietary alternative, at zero cost.

The ethical analysis did not produce simple answers, and that was the point. The question of who should profit from open-source labour, and whether all software should be open, does not have a clean resolution. But thinking through those questions honestly has made me more aware of the choices I will face as a developer: what I build, how I licence it, and whether I give back to the communities whose work makes mine possible.

Every tool used to write and test the shell scripts in this project — bash, the Linux kernel, the text editor, the package manager — was itself open source. That is not incidental. It is the point. Open source is the foundation, not the footnote.

6. References

The following sources were used for research and reference. All written analysis is in my own words.

- van Rossum, G. (1991). Python 0.9.0 release on alt.sources newsgroup. python.org/doc/essays/foreword
- Python Software Foundation. (2024). PSF Licence Version 2. python.org/psf/licence
- Python Software Foundation. (2004). PEP 20 — The Zen of Python. peps.python.org/pep-0020
- Raymond, E. S. (1999). The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Stallman, R. M. (2024). The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. (2024). The Open Source Definition. opensource.org/osdw
- Python Software Foundation. (2024). History and Licence. docs.python.org/3/licence.html
- Shotts, W. E. (2019). The Linux Command Line, 5th ed. No Starch Press. linuxcommand.org
- GNU Project. (2024). GNU Bash Manual. gnu.org/software/bash/manual
- SPDX. (2024). SPDX Licence List. spdx.org/licences
- CPython source code repository. github.com/python/cpython
- van Rossum, G. (2002). Python — computer programming language. Encyclopaedia Britannica.